

Función lambda, funciones fábrica y decoradores

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

Repaso: función lambda

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
  
help(list.sort)
```

```
sort(self, /, *, key=None, reverse=False)
```

Sort the list in ascending order and return None.

The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values.

The reverse flag can be set to sort in descending order.

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')  
datos.sort(key=lambda tupla: tupla[1])  
print(datos)
```

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')  
datos.sort(key=lambda tupla: tupla[1])  
print(datos)
```

tupla[0] tupla[1]

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]
datos.sort()
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')
datos.sort(key=lambda tupla: tupla[1])
print(datos)
```

tupla[0] tupla[1]

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]
datos.sort()
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')
datos.sort(key=lambda tupla: tupla[1])
print(datos)
```

tupla[0] tupla[1]

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]  
datos.sort()  
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')  
datos.sort(key=lambda tupla: tupla[1])  
print(datos)
```

key = 25

tupla[0] tupla[1]

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]
datos.sort()
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')
datos.sort(key=lambda tupla: tupla[1])
print(datos)
```

key = 25

key = 19

tupla[0] tupla[1]

```
datos = [("Guadalupe", 25), ("Luis", 19), ("Carlota", 31)]
datos.sort()
print(datos)
```

```
[('Carlota', 31), ('Guadalupe', 25), ('Luis', 19)]
```

¿Cómo
ordenamos por
edad?

```
print('ordenar por edad:')
datos.sort(key=lambda tupla: tupla[1])
print(datos)
```

key = 25

key = 19

Ejercicios:

- Ordena la lista tomando como criterio la segunda letra de cada palabra, en vez de la primera.
- Ordena la lista tomando como criterio la última letra de cada palabra.
- Ordena la lista tomando como criterio la longitud de cada palabra.
- Ordena tomando como criterio el segundo dígito de la edad de cada persona

```
print(f'\n{"Ordenar por edad":*^30}')
datos.sort(key=lambda tupla: tupla[1])
print(datos)

print(f'\n{"Ordenar por segunda letra":*^30}')
datos.sort(key=lambda tupla: tupla[0][1])
print(datos)

print(f'\n{"Ordenar por última letra":*^30}')
datos.sort(key=lambda tupla: tupla[0][-1])
print(datos)

print(f'\n{"Ordenar por longitud":*^30}')
datos.sort(key=lambda tupla: len(tupla[0]), reverse=True)
print(datos)

print(f'\n{"Ordenar por segundo dígito":*^30}')
datos.sort(key=lambda tupla: str(tupla[1])[-1])
print(datos)
```

Repaso: función lambda

```
def multiplicador(n):  
    return lambda x: x * n  
  
por_tres = multiplicador(3)  
print(por_tres(10))
```

Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

```
suma3 = fabrica_suma(3)
```

¿A qué hace
referencia suma3?

Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

```
suma3 = fabrica_suma(3)
```

¿A qué hace
referencia suma3?

Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

```
suma3 = fabrica_suma(3)
```

```
suma3(5)  ????
```

¿A qué hace
referencia suma3?

Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

```
suma3 = fabrica_suma(3)
```

```
suma3(5)  ????
```

closure: la función suma3 recuerda el valor de N. En este caso N=3

¿A qué hace referencia suma3?

Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

closure: la función suma3 recuerda el valor de N. En este caso N=3

```
suma3 = fabrica_suma(3)
```

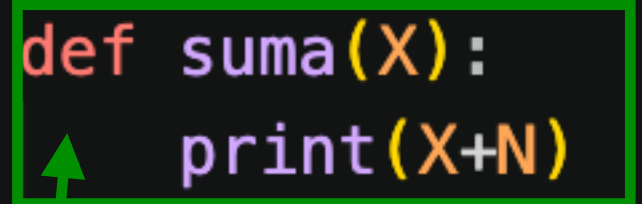
¿A qué hace referencia suma3?

```
suma3(5)
```

Se ejecuta únicamente la función suma

Factory functions

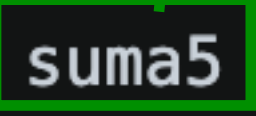
```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```



```
suma3 = fabrica_suma(3)
```

```
suma3(5)
```

```
suma5 = fabrica_suma(5)  
suma5(5)
```



Factory functions

```
def fabrica_suma(N):  
    def suma(X):  
        print(X+N)  
    return suma
```

```
suma3 = fabrica_suma(3)
```

```
suma3(5)
```

```
suma5 = fabrica_suma(5)  
suma5(5)
```

```
# notación función  
fabrica_suma(3)(10)  
fabrica_suma(3)(20)
```

https://gitlab.com/lucyhimura.86/python-para-ciencias-de-la-atmosfera/-/blob/master/Clases/clase5.ipynb?ref_type=heads

Decoradores

- Un decorador es una función que envuelve a otra función para modificar o extender su comportamiento sin cambiar su código.

```
def saludo():  
    print('Hola mundo')  
  
saludo()
```

Decoradores

```
def saludo():  
    print('Hola mundo')  
  
def decorador(fun):  
    def envolvente():  
        print('Antes de ejecutar mi función')  
        fun()  
        print('Después de ejecutar mi función')  
    return envolvente  
  
wrapper = decorador(saludo)  
wrapper()
```

Decoradores

```
def saludo():  
    print('Hola mundo')  
  
def decorador(fun):  
    def envolvente():  
        print('Antes de ejecutar mi función')  
        fun()  
        print('Después de ejecutar mi función')  
    return envolvente  
  
wrapper = decorador(saludo)  
wrapper()
```

Ejercicio:

- Hacer un decorador para una función suma

Decoradores

```
def saludo():  
    print('Hola mundo')  
  
def decorador(fun):  
    def envolvente():  
        print('Antes de ejecutar mi función')  
        fun()  
        print('Después de ejecutar mi función')  
    return envolvente
```

```
wrapper = decorador(saludo)  
wrapper()
```

```
def decorador(fun):  
    def envolvente():  
        print('Antes de ejecutar mi función')  
        fun()  
        print('Después de ejecutar mi función')  
    return envolvente
```

```
@decorador  
def saludo():  
    print('Hola mundo')
```

```
saludo()
```

Ejemplos

```
def decorador(fun):  
    def envolvente(*args):  
        print('Antes de ejecutar mi función')  
        fun(*args)  
        print('Después de ejecutar mi función')  
    return envolvente
```

```
@decorador  
def suma(*args):  
    print(sum(args))
```

```
suma(1,2,3,4,4)
```

```
@decorador  
def maximo(*args):  
    print(max(args))
```

```
maximo(1,33,45,3,65,788)
```

```
@decorador  
def minimo(*args):  
    print(min(args))
```

```
minimo(1,32,21,232,121,3)
```